# Informe técnico — Pedro, asistente de compras

**Curso:** Agentes Cognitivos com LangChain (Infnet). **Actualizado:** 2026-11-11. **Modelo:** gpt-4.1-mini. **Pruebas automáticas:** 142, todas en verde.

Este informe explica cómo el proyecto responde a las 29 rúbricas de la guía integrada (R01–R29). Cada rúbrica tiene aquí un resumen; la respuesta completa, con sus tablas y sus límites, está en docs/respuestas_rubricas.md. Las ejecuciones con el modelo real están guardadas en docs/trazas/ y el registro de lo ejecutado, en docs/evidencias.md.

## 1. Resumen

Pedro es un asistente que ayuda a comprar repuestos de electrodomésticos: compara las ofertas de varios proveedores para una pieza y una cantidad, y recomienda una. Todos los datos son sintéticos.

- **Qué hace bien.** Compara proveedores con reglas exactas, mantiene una conversación de varios turnos, responde preguntas sobre políticas con su fuente, confirma un código postal con una API real y no publica nada sin verificarlo.
- **Resultado medido.** En la evaluación vigente en LangSmith acierta el ganador y el importe en 14 de 14 recomendaciones y las 24 propuestas del modelo pasan la verificación a la primera.
- **Cómo se trabajó.** Cada rúbrica tiene una ejecución guardada. Varias ejecuciones con el modelo real destaparon fallos que las pruebas no veían; se diagnosticaron, se corrigieron y se volvieron a medir, y el antes y el después se conservan.
- **Qué queda abierto.** El vídeo de demostración, y tres supuestos que debe confirmar el profesor: el índice FAISS, los casos de evaluación y la escala de madurez.

## 2. Problema y dominio

Un comprador necesita una pieza, en cierta cantidad, a veces con un presupuesto y un plazo. Varios proveedores la ofrecen con distinto precio, flete, stock, plazo y pedido mínimo. El asistente debe decir cuál conviene y por qué las demás no.

El catálogo es una base SQLite sintética y reproducible (semilla 20261107): 2.640 piezas, 7.920 ofertas, 7.920 compatibilidades y 47.520 precios históricos. Cada conversación trabaja sobre un escenario aislado: una copia de la base con un identificador propio y un manifiesto con su huella. El asistente solo lee: no compra ni paga.

## 3. El núcleo: comparar proveedores

Todo lo demás existe para que esta comparación sea correcta y se pueda confiar en ella.

### Reglas

| Regla | La oferta se descarta si… |
|---|---|
| Stock | No hay unidades suficientes |
| Pedido mínimo | El proveedor exige comprar más unidades de las pedidas |
| Plazo | Tarda más que el plazo máximo del usuario, si lo dio |
| Presupuesto | El total supera el presupuesto del usuario, si lo dio |

El total es cantidad por precio unitario más flete, siempre en centavos enteros. Las ofertas se ordenan así: primero las que cumplen; entre ellas, las del origen que prefiere el usuario (nacional o internacional), si guardó esa preferencia; después por total, plazo y proveedor. La preferencia nunca hace ganar a una oferta que no cumple, y si respetarla cuesta más, la respuesta lo dice. Cada respuesta muestra todas las ofertas, con su origen y el motivo de descarte.

### El hallazgo que cambió la comparación

Durante casi todo el proyecto la comparación ignoró el pedido mínimo, aunque el dato estaba en el catálogo. El ejemplo de todas las demostraciones, 2 unidades de PEC-01-0001, recomendaba a CentroPeças por R$ 2.963,07, y ese proveedor exige 5 unidades. Hay 1.151 ofertas con stock y pedido mínimo mayor que 2. Ninguna prueba ni métrica lo detectó: la referencia de evaluación tenía el mismo hueco, así que el agente y la referencia coincidían. Se encontró al revisar qué datos del proveedor usaba la comparación. Al corregirlo cambiaron 5 de los 14 casos de recomendación de la evaluación.

### Historial y aviso de precio

El catálogo guarda seis precios mensuales por oferta. Para cada oferta válida la respuesta sitúa el precio de hoy: el del mes anterior, la media de los cinco meses previos y si está en su punto más bajo o más alto. El historial informa y no decide: gana el menor total de hoy.

Cuando el precio de hoy se aparta al menos un 10 % de su media, se añade un aviso. Ese aviso reúne las técnicas del curso sobre el problema central:

| Paso | Técnica | Rúbrica |
|---|---|---|
| Señal del precio, hechos y aviso de plantilla | Funciones hardcoded | R19 |
| Consejo redactado por el modelo | Prompt PTCF con few-shot | R11 |
| El consejo llega como objeto validado | Output parser Pydantic | R04 |
| Aviso y consejo se calculan en dos ramas, con plan B | RunnableParallel y with_fallbacks | R06, R09, R12 |
| El consejo solo se publica si sus cifras salen de los hechos y el NLI lo respalda | Verificación exacta y NLI | R24 |

Con el modelo y el NLI reales, sobre quince piezas, pasaron la verificación 5 consejos con el primer prompt y 14 con el prompt ajustado. El modelo añadía valoraciones («una buena oportunidad») imitando un ejemplo few-shot, y el NLI las rechazaba. Es aquí, verificando texto libre, donde el modelo NLI hace un trabajo que el cálculo exacto no puede hacer.

## 4. Arquitectura

Un turno sigue siempre el mismo recorrido: interpretar el mensaje, decidir el camino, consultar, verificar y responder.

- **Interpretar.** Las reglas identifican la pieza, que tiene formato fijo; un intérprete con modelo decide si el mensaje es una compra y qué cambia. Si el mensaje trae varias intenciones, un planificador lo divide en peticiones.
- **Decidir.** Lo que se reconoce sin ambigüedad va por una ruta fija (listado, ficha, aclaración). El resto va al agente.
- **Consultar.** El agente, construido con create_openai_tools_agent y AgentExecutor, elige entre seis herramientas: tres de SQL, una de RAG, una de API y una de memoria.
- **Verificar.** Toda propuesta pasa una verificación exacta y un modelo NLI. Antes, un paso de anclaje rellena con datos de las herramientas lo que el modelo copiaría mal.
- **Responder.** Solo se guarda en memoria la pregunta y la respuesta publicada.

Un principio recorre el diseño: **el modelo decide qué responder; los datos los pone el sistema.** Importes, ganador, citas, ciudad de un CEP y tablas de resultados salen siempre de las herramientas, nunca de lo que escribe el modelo.

La interfaz es Streamlit (app.py) y toda la lógica vive en src/pedro/. Sin clave de OpenAI la aplicación funciona en modo degradado con las rutas fijas.

## 5. Fase 1 — Chain ejecutable y arquitectura (R01–R06)

| Rúbrica | Qué se hizo | Resultado principal |
|---|---|---|
| R01 | Anatomía, tipo y madurez del agente | Agente híbrido; madurez nivel 4 demostrada |

| Rúbrica | Qué se hizo | Resultado principal |
|---|---|---|
| R02 | Planificación y memoria aumentada | Cinco pasos observables y planificador de turno |
| R03 | Plantillas de prompt reutilizables | Nueve ChatPromptTemplate con nombre propio |
| R04 | Salida estructurada en JSON | Laboratorio de ocho salidas rotas |
| R05 | invoke, batch, stream y @chain | Los tres medidos con el modelo real |
| R06 | LCEL y justificación | Los dos estilos comparados por lo que deja ver la traza |

## R01 — Anatomía, tipo y madurez

La anatomía se lee de los objetos reales del agente, no se escribe a mano: modelo, ejecutor, prompt PTCF, seis herramientas, memoria, base de conocimiento y puerta de publicación. Pedro es un agente **híbrido**: una capa reactiva de reglas resuelve lo que no debe quedar a criterio del modelo, y una capa deliberativa elige herramientas, mantiene estado y revisa su propuesta. La madurez se sitúa en el nivel 4 de una escala genérica de cinco (agente con memoria y autoverificación) y está demostrada con mediciones. No se busca el nivel 5: el problema no requiere varios agentes ni ejecutar compras. La escala es un supuesto declarado, porque la de la primera clase no está disponible.

## R02 — Planificación y memoria aumentada

Una solicitud se divide en cinco pasos con salida observable: extraer, consultar, comparar, verificar y publicar. Un mensaje con varias intenciones pasa antes por un planificador de turno: el modelo escribe un plan con una petición por intención, el plan se valida (no puede añadir datos que el usuario no escribió) y cada petición se resuelve y verifica por separado. Con el modelo real, un mensaje que mezclaba compra, política y CEP produjo tres peticiones, las tres publicadas; antes del planificador no se publicaba nada. La memoria se reparte en dos niveles: la sesión guarda lo que solo sirve para la compra en curso y el usuario guarda lo que debe sobrevivir. Precios y stock no son memoria: siempre se releen.

## R03 — Plantillas de prompt reutilizables

Las plantillas están reunidas en src/pedro/prompts.py, todas ChatPromptTemplate: extracción (en tres variantes), reparación, text-to-SQL, planificador, intérprete, consejo de precio y agente. La de extracción se reutiliza en la cadena declarativa y en la imperativa, y se ejecutó con el modelo real sobre tres solicitudes del dominio. Se eligió ChatPromptTemplate porque separa las reglas (mensaje de sistema) de lo que escribe el usuario (mensaje humano).

## R04 — Salida estructurada en JSON

La cadena convierte una solicitud en un objeto validado con Pydantic. Para probar su robustez se construyó un laboratorio con ocho salidas típicas de un modelo; cada defecto lo resuelve una capa distinta: el parser (cercas de markdown), una segunda cadena que repara (JSON cortado, tipos incorrectos, texto libre) o una regla de dominio (los faltantes los calcula el código). Con el modelo real como reparador, las cinco reparaciones devolvieron los datos correctos. El laboratorio reveló que el parser acepta JSON truncado y pierde un dato sin avisar; la cadena robusta lo comprueba. También se ejecutó with_structured_output, con resultado idéntico; se mantiene el parser porque deja ver y reparar la salida cruda.

## R05 — Runnable y composición imperativa

Con el modelo real: invoke produce un objeto validado; batch procesó tres solicitudes en 1,43 s frente a 4,55 s de tres invoke en serie; stream entregó 19 estados parciales, el primero a los 0,45 s. Lo aprendido: stream solo transmite si todos los pasos saben transmitir, y con el parser Pydantic llega un único fragmento. La composición imperativa usa @chain en dos cadenas; una repite con try/except la reparación de R04 y da el mismo resultado en el laboratorio. Son mediciones únicas, no un benchmark.

## R06 — Composición declarativa y elección

Hay cuatro cadenas LCEL (secuencia, fallback, stream parcial y paralelo). La cadena con reparación existe en los dos estilos y da el mismo resultado; lo que cambia es lo observable: en la versión declarativa la traza registra 16 pasos cuando hay reparación, incluido el error, y en la imperativa 9, sin el error. Decisión: LCEL

para lo que es una tubería, e imperativo para decidir el camino de cada turno, que tiene ramas y salidas anticipadas.

## 6. Fase 2 — Agente con herramientas y ReAct (R07–R12)

| Rúbrica | Qué se hizo | Resultado principal |
|---|---|---|
| R07 | Comparación de orquestadores | Seis comparados; LangChain y LangGraph ejecutados |
| R08 | ReAct con initialize_agent | Responde bien con memoria y falla sin ella |
| R09 | Chains y RouterChain | Router por reglas 2 de 7; router LLM 6 de 7 |
| R10 | API externa por function calling | ViaCEP real en tres casos |
| R11 | PTCF, few-shot y chain-of-thought | Zero-shot 6 de 10; few-shot 10 de 10 |
| R12 | Componentes de confiabilidad | Los cuatro, con caso normal y de error |

### R07 — Orquestadores

Se comparan LangChain, LangGraph, LlamaIndex, AutoGen, Semantic Kernel y Langflow por autonomía, abstracción y modularidad. El mismo agente pequeño se ejecutó en LangChain y en LangGraph: dieron las mismas respuestas, pero LangGraph guardó en su estado el resultado de una herramienta y no la volvió a llamar. Para Pedro, donde precios y stock deben releerse, esa diferencia importa. LangChain se mantiene porque el proyecto usa 13 tipos de componentes suyos y porque la rúbrica nombra sus APIs. Tres de ellas están marcadas como obsoletas; la migración natural sería a LangGraph. Los otros cuatro orquestadores se comparan de forma conceptual.

### R08 — ReAct

Un agente con initialize_agent y ZERO_SHOT_REACT_DESCRIPTION, dos herramientas y ConversationBufferMemory se ejecutó con el modelo real y dejó escrito su ciclo de pensamiento, acción y observación. Lo aprendido: el prompt zero-shot original no lee la memoria. Con el historial dentro del prompt resolvió una pregunta de seguimiento (CentroPeças, 2 días); sin él, pidió de nuevo la pieza.

### R09 — Chains y routing

Hay cadenas secuenciales y una paralela, y dos routers sobre RouterChain. En siete preguntas, el router por reglas acertó 2 rutas y el decidido por el modelo, 6. Las preguntas se eligieron para poner a prueba las reglas. El chat no usa ninguno como puerta de entrada: el agente ya enruta al elegir herramienta.

### R10 — API externa

El agente llama a ViaCEP, una API pública real, por function calling. Están guardados el esquema que recibe el modelo, los argumentos que envía, la respuesta de la API y lo publicado, en tres casos: válido (São Paulo/SP), inexistente e inválido. La primera ejecución no publicó la ciudad porque el modelo copiaba mal un identificador; ahora la respuesta se rellena con lo que devolvió la herramienta.

### R11 — Prompts

El prompt del agente sigue PTCF (Persona, Tarea, Contexto, Formato) y lleva ejemplos few-shot. Se midieron tres técnicas en diez solicitudes difíciles: sin ejemplos 6 aciertos, con few-shot 10 y con few-shot y chain-of-thought 10. El razonamiento se escribe en un campo antes que los datos, de modo que es observable. En el agente, el pensamiento paso a paso son sus llamadas a herramientas, que quedan registradas. El aviso de precio añadió otra lección: el modelo imita el estilo del ejemplo, no solo su formato.

### R12 — Confiabilidad

Los cuatro componentes tienen un caso normal y uno de error: parser con reparación, memoria que guarda solo lo publicado, búsqueda FAISS con filtro y un callback que registra cada herramienta y cada verificación. Ese registro es lo que permitió encontrar los fallos del agente con el modelo real.

## 7. Fase 3 — Chatbot con memoria (R13–R18)

| Rúbrica | Qué se hizo | Resultado principal |
|---|---|---|
| R13 | Memorias según CoALA | Las cuatro, con su contenido tras una conversación real |
| R14 | Ventana y búsqueda vectorial | Ventana de 12 sobre 16 mensajes; tres recuerdos recuperados |
| R15 | Herramientas de memoria | LangMem ejecutado frente a la memoria propia |
| R16 | ConversationBufferMemory | Chatbot de cuatro turnos que recuerda todo |
| R17 | Intención y entidades | Reglas solas 8 de 14; reglas y modelo 14 de 14 |
| R18 | Conversación real multi-turno | Ocho turnos con el estado correcto |

### R13 — Memorias

Pedro tiene las cuatro memorias de CoALA: de trabajo (mensaje, borrador de compra y ventana, por sesión), episódica (comparaciones pasadas con vector, por usuario), semántica (preferencias, catálogo y políticas) y procedimental (prompt y reglas). Una preferencia es un hecho y se aplica: ordena las ofertas de cada comparación. Un episodio es un suceso y solo se recuerda, porque sus datos pueden haber caducado.

### R14 — Ventana de contexto

El agente recibe una ventana deslizante de los últimos 12 mensajes más los tres episodios más parecidos a la pregunta. Tras una conversación real había 16 mensajes guardados y 12 en la ventana; en una sesión nueva se recuperaron tres episodios (similitud entre 0,21 y 0,35) y ninguno para otro usuario. Se eligió ventana y no resumen porque la ventana no puede alterar una cifra.

### R15 — Herramientas de memoria

Se comparan Mem0, Letta/MemGPT y LangMem con la memoria propia. LangMem se ejecutó con la misma conversación: las dos guardaron la preferencia y aislaron por usuario. LangMem deja que el modelo decida qué recordar y supo responder qué recordaba; su agente, sin verificación, dio una respuesta contradictoria sobre el stock. Se mantiene la memoria propia porque lo que entra debe ser comprobable. Mem0 y Letta se comparan de forma conceptual.

### R16 — Chatbot con buffer

Un chatbot con ConversationChain y ConversationBufferMemory recordó en cuatro turnos el nombre, la cantidad, el presupuesto y el plazo. La memoria creció de 240 a 885 caracteres: el buffer no tiene límite, y por eso el agente final usa la ventana.

### R17 — Intención y entidades

El chat reconoce seis intenciones y cinco entidades. Las reglas identifican la pieza y un intérprete few-shot decide si el mensaje es una compra y qué cambia. En catorce mensajes difíciles, las reglas solas acertaron 8 y con el modelo 14: entiende «súbelo a media docena», «olvida el plazo» o «duplica la cantidad». Cuando falta la pieza o la cantidad, pregunta en vez de inventar.

### R18 — Conversación real

En ocho turnos con el modelo real el agente descartó una compra de 2 unidades por pedido mínimo, la recalculó con 5 conservando el plazo, cambió el plazo por un presupuesto, guardó una preferencia, respondió un CEP y una política sin tocar la compra, volvió a comparar con el estado correcto y aplicó la preferencia a otra pieza, donde el proveedor nacional ganó costando más. La primera versión de esta conversación falló en tres turnos; las dos trazas se conservan.

## 8. Fase 4 — SQL, APIs y RAG (R19–R23)

| Rúbrica | Qué se hizo | Resultado principal |
|---|---|---|
| R19 | Herramientas hardcoded y semánticas | Seis herramientas catalogadas |
| R20 | APIs por tipo de uso | Tres externas y cuatro internas |
| R21 | Text-to-query | 5 de 5 preguntas; 7 de 7 consultas hostiles rechazadas |

| Rúbrica | Qué se hizo | Resultado principal |
|---|---|---|
| R22 | Índice FAISS existente | El agente lo carga, no lo construye |
| R23 | Fallo de recuperación | Diagnosticado y corregido, con antes y después |

### R19 — Herramientas

Seis herramientas: cuatro de lógica fija, una semántica (políticas) y una mixta (catálogo). La regla de reparto: todo lo que es un número o una regla lo hace código fijo; lo semántico busca por significado y nunca decide un importe.

### R20 — APIs

Tres externas (ViaCEP para confirmar el destino, OpenAI como motor y LangSmith para evaluar) y cuatro internas (catálogo en solo lectura, políticas en FAISS, memoria aislada y verificación). El tipo de uso fija la integración: lo que lee es de solo lectura, lo que llama a un tercero tiene límite de tiempo y estados de error, y la verificación queda fuera del alcance del modelo. Ninguna API compra ni modifica el catálogo.

### R21 — Text-to-query

La herramienta de catálogo tiene dos modos. Con un ID de pieza usa plantillas fijas. Para preguntas abiertas hay un text-to-SQL completo: el modelo escribe un SELECT y el motor decide si lo ejecuta. Tres barreras: la forma (una sola sentencia SELECT), el autorizador de SQLite (solo las tablas y columnas de una lista) y los límites (solo lectura, 50 filas). Con el modelo real respondió bien 5 de 5 preguntas, y 3 de 5 antes de mejorar el prompt: dos consultas eran válidas pero estaban mal planteadas. Válida no significa correcta, y por eso el chat muestra siempre la consulta ejecutada. La tabla de resultados la arma el sistema.

### R22 — Índice FAISS existente

El índice se construye una vez, fuera del agente, y se guarda con un manifiesto. El agente solo lo carga, en 0,03 s. Como su almacén es un archivo pickle, la carga exige un manifiesto completo, huellas que coincidan y confianza explícita; se probó que rechaza un archivo alterado. El índice es propio: la actividad no entregó ninguno.

### R23 — Fallo de recuperación

La pregunta «¿Se puede mandar en varias cajas?» no recuperaba la regla correcta, que está en el corpus. Diagnóstico: desajuste de vocabulario; el usuario dice «varias cajas» y la política dice «entrega parcial». Corrección: reformular la consulta con el término del dominio. Con el mismo índice y los mismos cuatro resultados, el fragmento pasó de no aparecer a la primera posición.

## 9. Fase 5 — Agente final verificado y evaluado (R24–R29)

| Rúbrica | Qué se hizo | Resultado principal |
|---|---|---|
| R24 | Verificación factual y lógica | Ninguna recomendación contraria a la regla publicada |
| R25 | Agente único | SQL, RAG y API bajo el mismo ejecutor |
| R26 | create_openai_tools_agent y AgentExecutor | Ejecutado en conversación, evaluación e interfaz |
| R27 | RunnableWithMessageHistory | Solo se guarda lo publicado |
| R28 | Streamlit | La app ejecutada con el agente real |
| R29 | LangSmith | Ocho experimentos con seis métricas |

### R24 — Verificación

Ninguna respuesta se muestra sin pasar dos puertas dentro del flujo. La verificación exacta recalcula la respuesta con los datos del escenario: total, stock, pedido mínimo, presupuesto, plazo, ganador, identificadores y procedencia. El modelo NLI clasifica la relación entre evidencia y afirmación en tres clases, y solo entailment permite publicar. Un paso previo de anclaje rellena con datos de las herramientas las citas, el CEP, las preferencias, las cifras de la comparación y las tablas del catálogo. Si la verificación falla hay una

corrección; si vuelve a fallar, no se publica. Límite importante: la verificación comprueba contra la regla de negocio, no la regla; mientras la comparación ignoró el pedido mínimo, aprobó recomendaciones que un proveedor no aceptaría.

### R25 y R26 — Agente único y ejecutor

Un solo agente, construido con create_openai_tools_agent y AgentExecutor (seis iteraciones como máximo), reúne herramientas de SQL, RAG y API. En la conversación de ocho turnos usó las tres categorías con el mismo ejecutor, la misma memoria y el mismo verificador.

### R27 — Historial

El agente está envuelto en RunnableWithMessageHistory, con el historial en SQLite por usuario y sesión. Solo se guarda lo publicado: tras ocho turnos había 16 mensajes, ocho del usuario y ocho respuestas. El historial sobrevive a recrear el flujo.

### R28 — Interfaz

app.py conecta Streamlit al mismo flujo. Ejecutada con la herramienta de pruebas de Streamlit y el modelo real, respondió cuatro turnos de chat, abrió un chat nuevo, reabrió la conversación anterior con sus 8 mensajes y mostró las vistas de catálogo y de comparación sin errores. El chat y el formulario dieron el mismo importe para la misma compra (R$ 8.718,75 por 6 unidades).

### R29 — Evaluación con LangSmith

El flujo final se evaluó sobre 30 casos derivados con seis métricas. Resultado vigente:

| Métrica | Resultado |
|---|---|
| Ganador correcto | 14 de 14 |
| Importe correcto | 14 de 14 |
| Contenido correcto | 16 de 16 |
| Herramienta decisiva | 23 de 23 |
| Primera propuesta aceptada | 24 de 24 |
| Errores de verificación | 0 |
| Trayectoria completa | 23 de 30 |

Conclusiones, cada una con su dato:

- **Se mantiene gpt-4.1-mini.** En igualdad de condiciones, gpt-4o-mini acertó 13 de 14 ganadores y 19 de 23 propuestas a la primera.
- **La verificación cumple.** Cuando un modelo propuso un ganador equivocado, se rechazó y no se publicó.
- **La evaluación encontró un defecto del agente** (el CEP no se publicaba) y **un error de método**: los experimentos compartían memoria, y con 73 recuerdos acumulados la trayectoria cayó de 25 a 15 de 30.
- **La trayectoria completa es una métrica inestable.** Varió entre 13 y 25 de 30 entre corridas, porque la referencia exige un paso que el agente a veces se salta con el mismo resultado.
- **Una métrica perfecta no garantiza una regla correcta.** Durante seis experimentos el agente obtuvo 14 de 14 recomendando ofertas que incumplían el pedido mínimo, porque la referencia compartía el error.

## 10. Lo que enseñaron las ejecuciones reales

Estos fallos solo aparecieron al ejecutar con el modelo real. Las pruebas automáticas ya pasaban.

| Fallo observado | Causa | Corrección |
|---|---|---|
| Las preguntas de política no se publicaban | El modelo citaba el fragmento casi literal | El sistema ancla la cita al texto de la fuente |
| La ciudad de un CEP no se publicaba | El modelo copiaba mal un identificador | La respuesta se rellena con la observación de la API |

| Fallo observado | Causa | Corrección |
|---|---|---|
| Una compra válida no se publicaba | El modelo transcribía mal el ganador | Las cifras las pone la herramienta |
| Un mensaje con varias intenciones no se publicaba | La respuesta admite un solo tipo | Planificador de turno |
| No entendía «que sean 3» ni «olvida el plazo» | Reglas de diálogo demasiado estrechas | Intérprete con modelo |
| El parser perdía un dato con JSON cortado | Tolerancia del parser | Comprobación estricta |
| Dos consultas SQL válidas daban mal el resultado | El modelo confundía ID y SKU | Formato de los valores en el prompt |
| La trayectoria cambiaba entre experimentos | Memoria compartida | Memoria nueva en cada experimento |
| Recomendaba ofertas que el proveedor no aceptaría | La comparación ignoraba el pedido mínimo | Regla nueva y casos regenerados |
| Los consejos de precio se rechazaban | Un ejemplo few-shot invitaba a valorar | Ejemplos que solo enuncian hechos |

## 11. Limitaciones

- **Supuestos sin confirmar.** El índice FAISS es propio, los 30 casos de evaluación son derivados y no los oficiales, y la escala de madurez es genérica. El enunciado original en portugués no está disponible para un cotejo final.
- **Mediciones de una sola corrida.** El modelo no es determinista; las diferencias de uno o dos casos no son concluyentes.
- **Pedro no conversa.** No responde «¿qué recuerdas de mí?» ni sabe contestar a un comentario sin petición.
- **La verificación comprueba que la respuesta sea cierta, no que venga al caso**, y comprueba contra la regla de negocio, no la regla.
- **El aviso de precio salta mucho** (en cuatro de cada diez recomendaciones); sus umbrales no están calibrados.
- **El modelo NLI no es exacto.** Rechazó un consejo que era cierto; falla hacia el lado seguro. Admite 512 tokens.
- **Comparaciones conceptuales.** LlamaIndex, AutoGen, Semantic Kernel, Langflow, Mem0 y Letta no se ejecutaron.
- **Sin credenciales** el chat interpreta solo con reglas, que son frágiles ante formas de hablar no previstas.
- **La comparación no reparte un pedido entre proveedores** ni calcula el flete según el destino.

## 12. Mejoras futuras

- Añadir un tipo de respuesta conversacional, para saludos, comentarios y «¿qué recuerdas de mí?».
- Migrar la orquestación del turno a LangGraph y conservar los componentes de LangChain.
- Calibrar los umbrales del aviso de precio y del anclaje de citas con datos.
- Revisar la referencia de trayectoria con los casos oficiales.
- Usar el CEP para estimar flete y plazo según el destino.
- Resumir el historial que sale de la ventana de 12 mensajes.

## 13. Cómo reproducir las evidencias

- Instalar: uv sync --python 3.13 --all-extras --locked
- Pruebas: uv run pytest -q

- Notebooks, sin credenciales: uv run python scripts/crear_notebooks.py y uv run python scripts/ejecutar_notebooks.py
- Ejecuciones con el modelo real (requieren OPENAI_API_KEY): los scripts scripts/demostrar_*.py, que escriben en docs/trazas/
- Evaluación (requiere además LANGSMITH_API_KEY): uv run python evals/evaluar_flujo.py
- Interfaz: uv run streamlit run app.py

Los notebooks usan dobles de prueba y lo dicen en cada celda. Las trazas de docs/trazas/ con sufijo «antes» o «sin plan» y los primeros seis experimentos de LangSmith corresponden a versiones anteriores del agente y se conservan como registro.

## 14. Pendientes de entrega

- Grabar el vídeo de demostración y verificar el enlace de Drive.
- Cotejar las rúbricas con el enunciado original en portugués.
- Confirmar con el profesor el índice FAISS, los casos de evaluación y la escala de madurez.